
Tutorial 02

INF1008 Data Structures and Algorithms

Name: Timothy Chia

Student ID: 2501530

1. Write an algorithm to merge two Singly Linked Lists. Assume that the data in each list are in ascending order. The algorithm should return one Singly Linked List, containing all the data in ascending order.

```
from typing import Optional

class Node:
    def __init__(self, data: int, next: Optional["Node"] = None) -> None:
        self.data: int = data
        self.next: Optional["Node"] = next

def merge_sorted_lists(
    a: Optional[Node], b: Optional[Node]
) -> Optional[Node]:
    # Merge two sorted singly linked lists into one sorted list.
    # Time: O(n + m), Space: O(1) extra (re-links existing nodes).

    dummy = Node(0)    # temporary starter node
    tail = dummy        # tail of the merged list

    while a is not None and b is not None:
        if a.data <= b.data:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next

    # append any remaining nodes
    tail.next = a if a is not None else b
    return dummy.next

def print_list(head: Optional[Node]) -> None:
    cur = head
    out: list[str] = []
    while cur is not None:
        out.append(str(cur.data))
        cur = cur.next
    print(" -> ".join(out) if out else "(empty)")
```

```

if __name__ == "__main__":
    list1 = build_list([3, 6, 6, 10, 45, 45, 50])
    list2 = build_list([2, 3, 55, 60])

    merged = merge_sorted_lists(list1, list2)
    print_list(merged)
# 2 -> 3 -> 3 -> 6 -> 6 -> 10 -> 45 -> 45 -> 50 -> 55 -> 60

```

2. A sorting method with “Big-Oh” complexity $O(n \log_{10} n)$ spends exactly 1 millisecond to sort 1,000 data items. If the time $T(n)$ is directly proportional to $n \log_{10} n$, i.e. $T(n) = c n \log_{10} n$, derive a formula for $T(n)$ given the time $T(N)$ for sorting N items, and estimate how long this method will sort 1,000,000 items.

Since $T(n) = c n \log_{10} n$ and $T(N) = c N \log_{10} N$, we can eliminate c :

$$\begin{aligned}
 c = \frac{T(N)}{N \log_{10} N} &\Rightarrow T(n) = \frac{T(N)}{N \log_{10} N} n \log_{10} n \\
 &= T(N) \cdot \frac{n \log_{10} n}{N \log_{10} N}.
 \end{aligned}$$

Here, $T(1000) = 1$ ms and $\log_{10}(1000) = 3$. For $n = 1,000,000$, $\log_{10}(1,000,000) = 6$. Therefore,

$$\begin{aligned}
 T(1,000,000) &= 1 \text{ ms} \cdot \frac{1,000,000 \cdot 6}{1,000 \cdot 3} \\
 &= 1 \text{ ms} \cdot 2000 \\
 &= 2000 \text{ ms} \\
 &= 2 \text{ s}.
 \end{aligned}$$

3. Assume that each expression below gives the processing time $T(n)$ spent by an algorithm for a problem of size n . Select the dominant term(s) having the steepest increase in n and specify the lowest Big-Oh complexity.

Expression	Dominant term(s)	Big-Oh complexity
$5 + 0.0001n^3 + 0.025n$	$0.0001n^3$	$O(n^3)$
$500n + 100n^{1.5} + 50n \log_{10} n$	$100n^{1.5}$	$O(n^{1.5})$
$\frac{n^2}{\log_2 n} + n(\log_2 n)^2$	$\frac{n^2}{\log_2 n}$	$O\left(\frac{n^2}{\log n}\right)$

Note: $O\left(\frac{n^2}{\log n}\right) \subset O(n^2)$, so it is the tighter (lower) upper bound.

4. Algorithms A and B spend exactly

$$T_A(n) = C_A n \log_2 n \quad \text{and} \quad T_B(n) = C_B n^2$$

microseconds, respectively, for a problem of size n . Find the best algorithm for processing $n = 2^{20}$ data items if: A spends 10 microseconds to process 1024 items and B spends 1 microsecond to process 1024 items.

Since $1024 = 2^{10}$ and $\log_2(1024) = 10$:

$$10 = C_A \cdot 1024 \cdot 10 \Rightarrow C_A = \frac{10}{10240} = \frac{1}{1024}.$$

$$1 = C_B \cdot 1024^2 \Rightarrow C_B = \frac{1}{1024^2} = \frac{1}{2^{20}}.$$

Now evaluate at $n = 2^{20}$ (so $\log_2 n = 20$):

$$\begin{aligned} T_A(2^{20}) &= \frac{1}{1024} \cdot 2^{20} \cdot 20 \\ &= 2^{10} \cdot 20 \\ &= 20480 \mu s \\ &= 20.48 \text{ ms}, \\ T_B(2^{20}) &= \frac{1}{2^{20}} \cdot (2^{20})^2 \\ &= 2^{20} \mu s \\ &= 1048576 \mu s \approx 1.05 \text{ s}. \end{aligned}$$

Conclusion: Algorithm A is faster for $n = 2^{20}$ items.